

My Jenkins Repo	1
Note	1
Articles	1
Create Simple Job	2
Add job	2
Build	3
Create Php Job from Jenkins container	4
Install php	4
Add new Job	5
Configure	5
Build	6
Simple Job from Docker container	7
Add Cloud Agent (Docker container)	7
Php Pipeline	12
Enable Docker	12
Create php container	12
Install git ftp	13
Define ftp credentials	13
Create New pipeline	13
Deploy	15
PHP Free Style	15
Subdomain jenkins-php-pipeline.tantonius.com	15
Define FTP Credentials	15
01 Initialize GIT for deployment	16
Git repo for deployment	16
Job: job_php_initialize_git	16
02 Test	18
03 Deploy	18
Migrate	19
Start build with CLI	19
Push Docker Image	20
Build images	20
Push image	21

My Jenkins Repo

<https://github.com/cptuulia/jenkins>

This repo is private. Make it public, to make these examples to work.

Note

The most realistic paragraph is **PHP Free Style**.

To understand it you may need to read other chapters first.

Articles

Jenkins cli commands

<http://localhost:8080/manage/cli/>

Short intro

<https://www.youtube.com/watch?v=6YZvp2GwT0A>

<https://github.com/devopsjourney1/jenkins-101>

Jenkins with php

<https://simplyexplained.com/courses/get-started-with-jenkins/>

pipeline

<https://medium.com/novai-devops-101/building-a-jenkins-pipeline-for-a-php-application-in-docker-c0144d8388de>

<https://medium.com/novai-devops-101/building-a-jenkins-pipeline-for-php-application-deployment-e47898d92926>

Create Simple Job

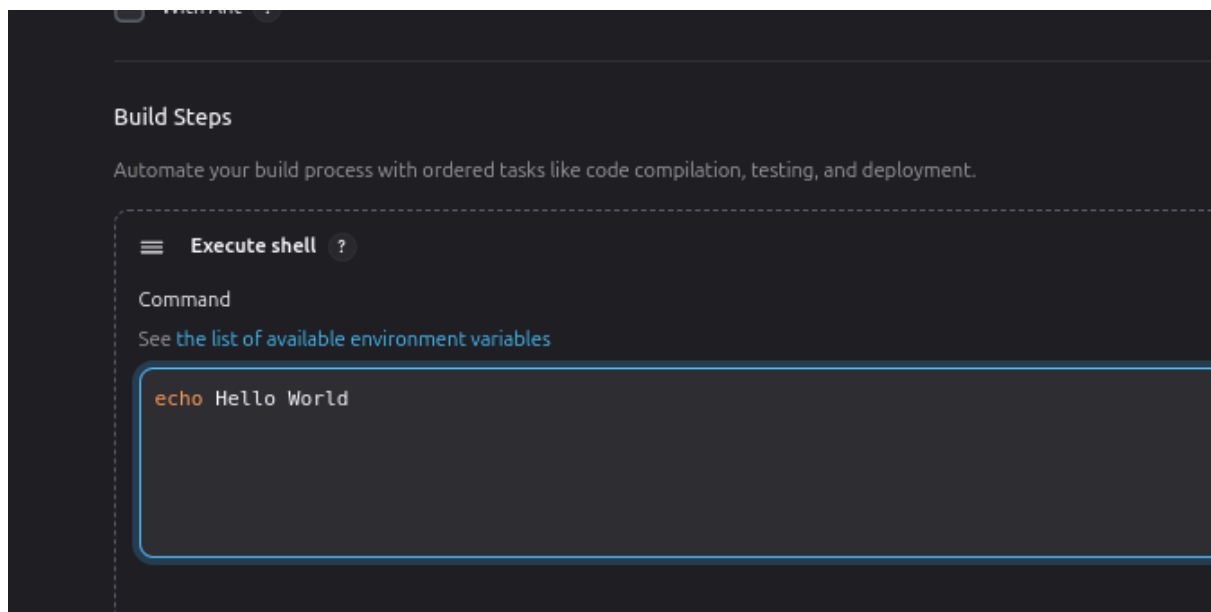
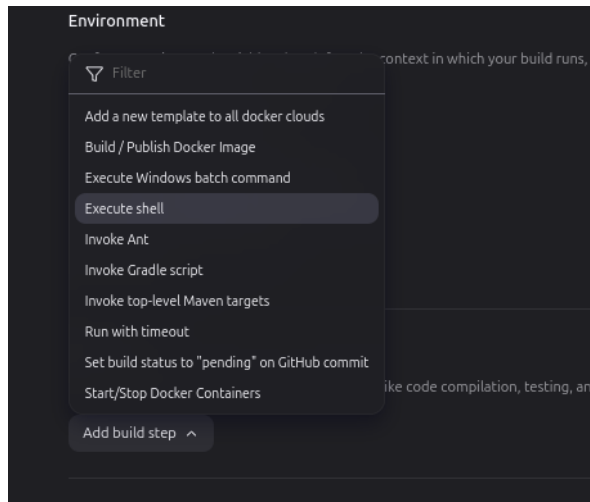
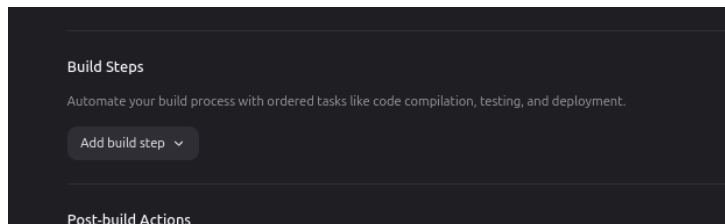
Here we just execute: Hello World on Jenkins container

Add job

<http://localhost:8080/view/all/newJob>

Simple Job as Freestyle project

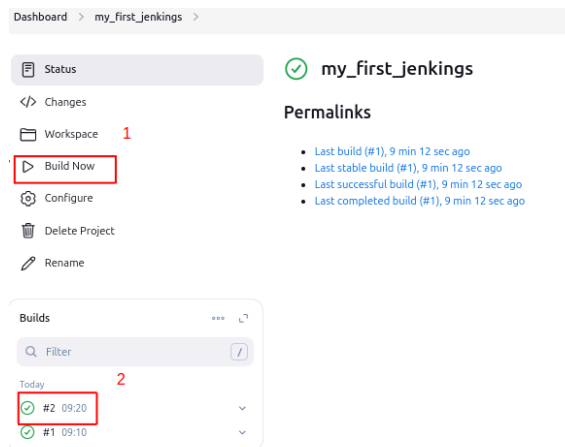
Leave everything as defaults, just add one Build step with a echo shell command



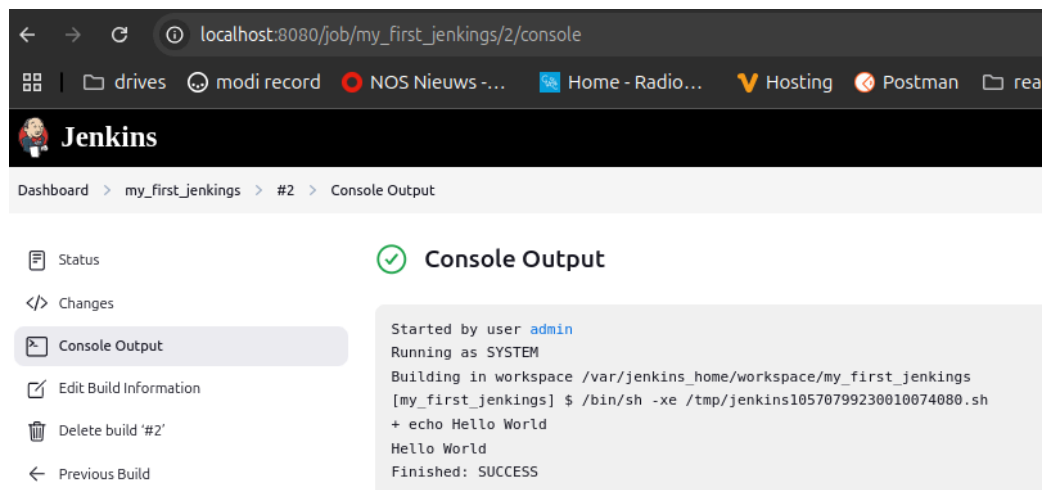
```
echo Hello World
```

Build

<http://localhost:8080/job/Simple%20Job/>



open http://localhost:8080/job/my_first_jenkins/2/console
(after the index #*, in this case 2)



Create Php Job from Jenkins container

In this example I install php directly to 'jenkins_local' container and run a php script

Install php

Install php on Jenkins container

```
# login as root
docker exec -it --user root jenkins_local sh
```

Check if we have php

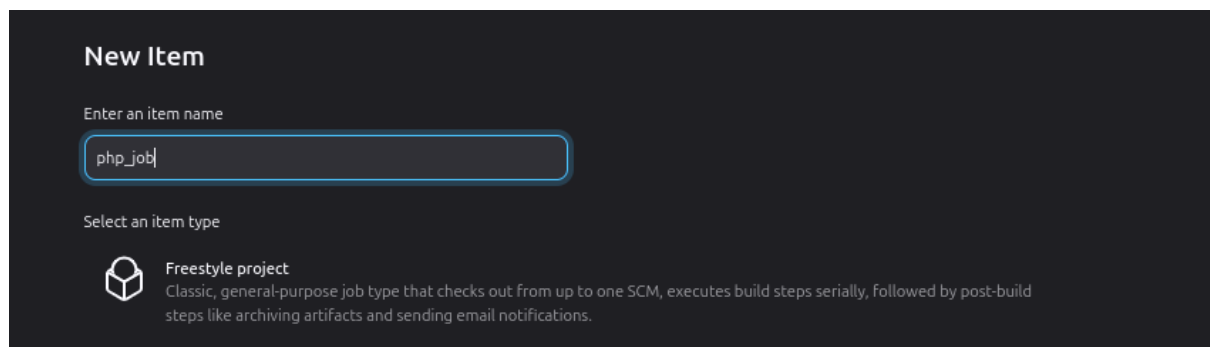
```
php -v
PHP 8.2.26 (cli) (built: Nov 25 2024 17:21:51) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.26, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.26, Copyright (c), by Zend Technologies
```

If not

```
apt-get update
apt-get install -y php
```

Add new Job

<http://localhost:8080/view/all/newJob>



New Item

Enter an item name

php_job

Select an item type

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Name : php_job
type: free style

Configure

Live all defaults , just add one build step, Execute shell:

Build Steps

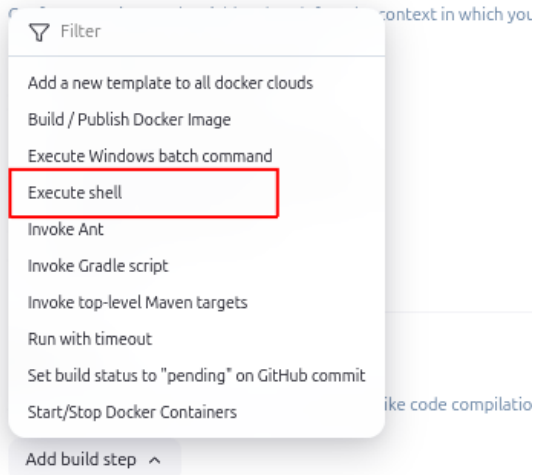
Automate your build process with ordered tasks like code compilation, testing, and deployment.

Add build step ▾

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other builds.

Environment



```
echo "#####"
echo php version
php -v
echo "#####"
echo Create file /var/jenkins_home/workspace/php_from_jenkins_container/test.php
echo "<?php phpinfo();>" >test.php

more test.php
echo "#####"
php test.php
echo "#####"
rm test.php
```

Build

You should see

Console Output

```
Started by user Tuulia Antonius
Running as SYSTEM
Building in workspace /var/jenkins_home/workspace/php_from_jenkins_container
[php_from_jenkins_container] $ /bin/sh -xe /tmp/jenkins12721239073464699210.sh
+ echo #####
#####
+ echo php version
php version
+ php -v
PHP 8.2.26 (cli) (built: Nov 25 2024 17:21:51) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.26, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.26, Copyright (c), by Zend Technologies
+ echo #####
#####
+ echo Create file /var/jenkins_home/workspace/php_from_jenkins_container/test.php
Create file /var/jenkins_home/workspace/php_from_jenkins_container/test.php
+ echo <?php phpinfo();
+ more test.php
:::::::::::::
test.php
:::::::::::::
<?php phpinfo();
+ echo #####
#####
+ php test.php
phpinfo()
PHP Version => 8.2.26

System => Linux 7f4a7e57edf0 6.8.0-52-generic #53-Ubuntu SMP PREEMPT_DYNAMIC Sat Jan 11 00:06:25 UTC 2025 x86_64
Build Date => Nov 25 2024 17:21:51
Build System => Linux
Server API => Command Line Interface
Virtual Directory Support => disabled
Configuration File (php.ini) Path => /etc/php/8.2/cli
Loaded Configuration File => /etc/php/8.2/cli/php.ini
Scan this dir for additional .ini files => /etc/php/8.2/cli/conf.d
Additional .ini files parsed => /etc/php/8.2/cli/conf.d/10-opcache.ini,
/etc/php/8.2/cli/conf.d/10-pdo.ini,
```

Simple Job from Docker container

Add Cloud Agent (Docker container)

Note there are bugs, it is better to enable Docker in the Jenkins container run sub container there. See in the later chapters...

It is better to use the containers without agents. See chapter Php Pipeline

1) For the first time we need to install the docker plugin. Others go to step 5

Open <http://localhost:8080/manage/cloud/>

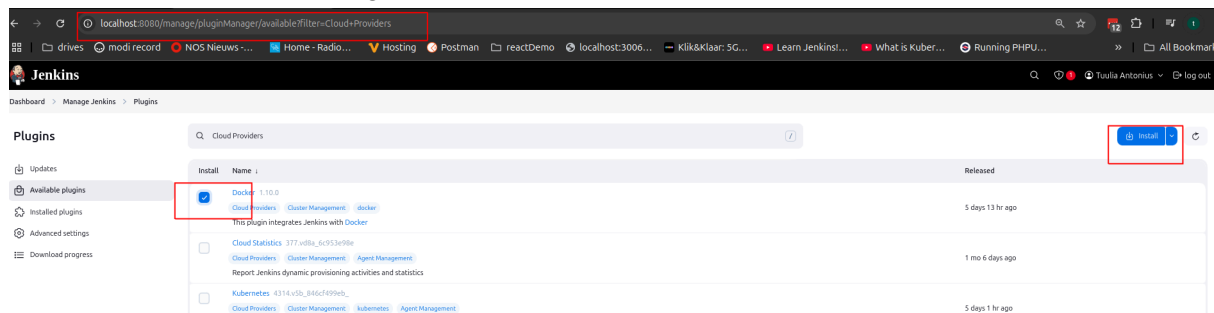
Clouds

There is no plugin installed that supports clouds.

[Install a plugin](#)

[Learn more about distributed builds](#)

- 2) Select Docker and click Install.
Check as in the second image below and restart Jenkins



Manage Jenkins > Plugins		
ns	Mina SSHD API :: Core	Success
lates	Gson API	Success
ilable plugins	Git client	Success
alled plugins	Git	Success
anced settings	GitHub	Success
ownload progress	GitHub Branch Source	Success
	Pipeline: GitHub Groovy Libraries	Success
	Pipeline Graph Analysis	Success
	Metrics	Success
	Pipeline Graph View	Success
	Git	Success
	EDDSA API	Success
	Trilead API	Success
	SSH Build Agents	Success
	Matrix Authorization Strategy	Success
	PAM Authentication	Success
	LDAP	Success
	Email Extension	Success
	Mailer	Success
	Theme Manager	Success
	Dark Theme	Success
	Loading plugin extensions	Success
	Cloud Statistics	Success
	Authentication Tokens API	Success
	Docker Commons	Success
	Apache HttpComponents Client 5.x API	Success
	Commons Compress API	Success
	Docker API	Success
	Docker	Success
	Loading plugin extensions	Success

- 5) Now we can make a new cloud
<http://localhost:8080/manage/cloud/new>

New cloud

Cloud name

Test

Type

☒ Docker

Create

6)

<http://localhost:8080/manage/cloud/create>

New cloud

Name ?

Test

Docker Cloud details ^ Edited

Docker Host URI ?

tcp://172.17.0.2:2375

Server credentials

- none - ▼

+ Add

Advanced ▼

Version = 27.5.1, API Version = 1.47

Test Connection

☒ Enabled ?

Error Duration ?

Default = 300

☐ Expose DOCKER_HOST ?

Container Cap ?

100

Name: Test

Where Host `tcp://172.17.0.2:2375`

Remember to TEST!

You get current host IP by

If jenkins_alpine_socat is not installed, install by

<https://github.com/cptuulia/jenkins/blob/main/README.md>

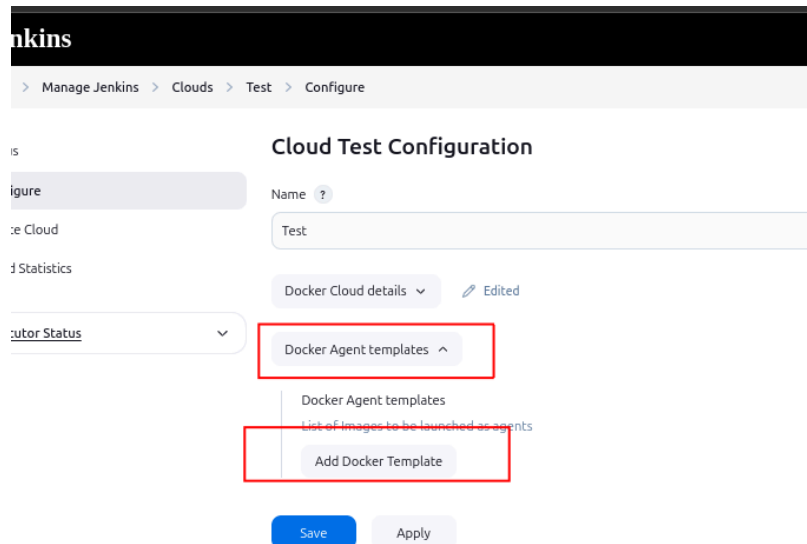
```
>docker inspect jenkins_alpine_socat |grep 'IPAddress'
"SecondaryIPAddresses": null,
"IPAddress": "172.17.0.2",
"IPAddress": "172.17.0.2",
```

You can test by

```
nc -zv 172.17.0.2 2375
```

```
Connection to 172.17.0.2 2375 port [tcp/*] succeeded!
```

7) As next we add the Docker Agent Template
<http://localhost:8080/manage/cloud/Test/configure>



Cloud Test Configuration

Name ?
Test

Docker Cloud details ▾ Edited

Docker Agent templates ^

Docker Agent templates
List of Images to be launched as agents

Add Docker Template

Save Apply



Docker Agent templates

Labels ?
jenkins-agent

Enabled ?
☒

Name ?
jenkins-agent

Docker Image ?
jenkins/agent

Registry Authentication ▾

Container settings ▾

Instance Capacity ?
1

Remote File System Root ?

Usage ?
Use this node as much as possible ▾

Idle timeout ?
10

Add Docker Template

Label and name: jenkins-agent

Enabled: Checked

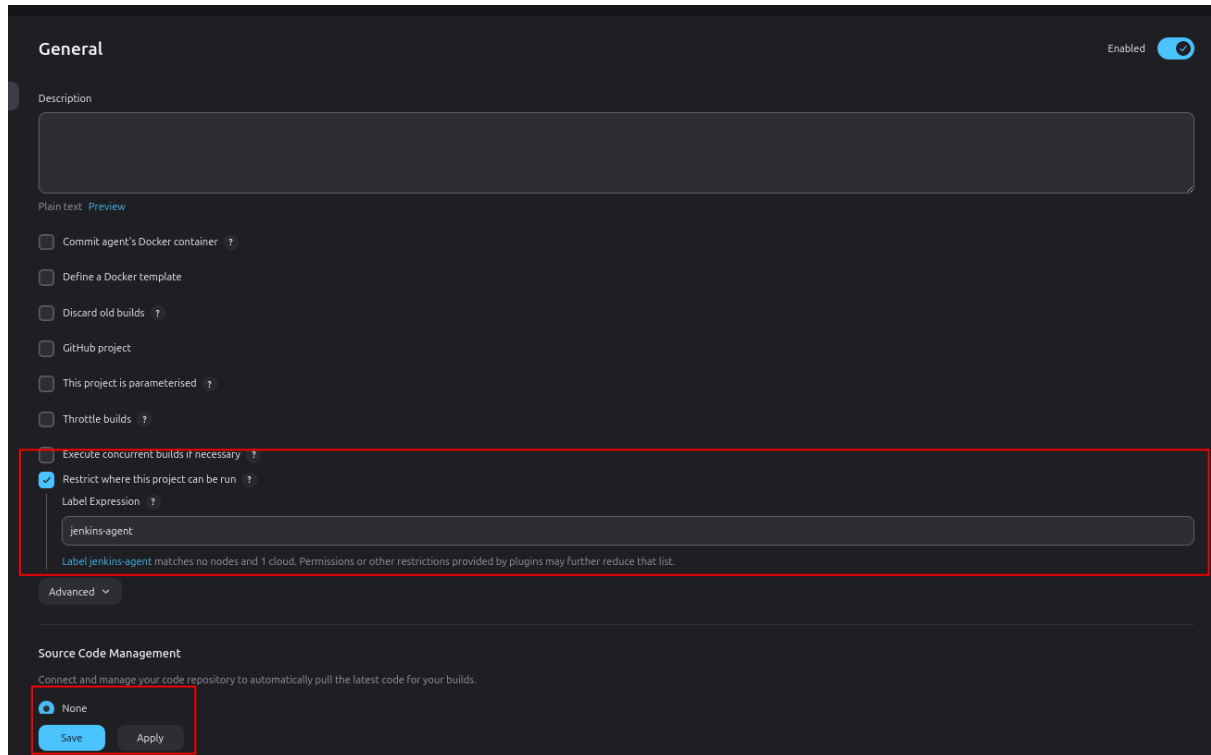
Docker image: `jenkins/agent` (this is the standard agent)

Instance Capacity: 1

Use Cloud Agent in Simple Job

Edit 'Simple Job' (made in the previous chapter) by defining the Cloud agent

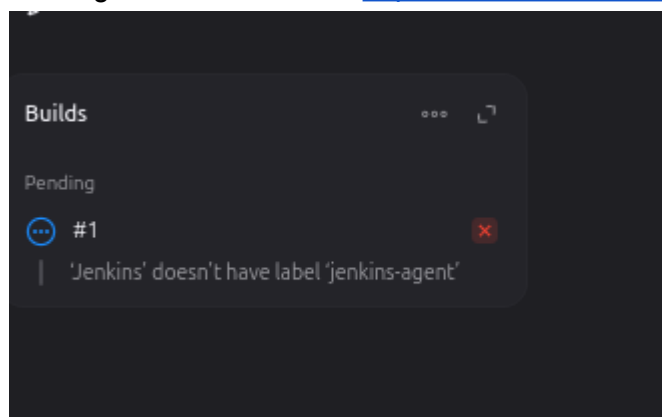
<http://localhost:8080/job/Simple%20Job/configure>



The screenshot shows the Jenkins configuration page for a job. The 'General' tab is selected. The 'Description' field is empty. The 'Plain text' section has a 'Preview' link. Below this, there are several checkboxes: 'Commit agent's Docker container', 'Define a Docker template', 'Discard old builds', 'GitHub project', 'This project is parameterised', 'Throttle builds', and 'Execute concurrent builds if necessary'. The 'Restrict where this project can be run' checkbox is checked. Below it, the 'Label Expression' is set to 'jenkins-agent'. A note below the label expression states: 'Label jenkins-agent matches no nodes and 1 cloud. Permissions or other restrictions provided by plugins may further reduce that list.' The 'Advanced' section is collapsed. The 'Source Code Management' section shows 'None' selected. The 'Save' button is highlighted.

Label: `jenkins-agent`

You might see some errors <http://localhost:8080/manage/log/all>



```
jenkins-agent

[Enabled]

Note: Disabled 3 min 23 sec ago due to error. Will re-enable in 1 min 36 sec.
Reason: Template provisioning failed.

com.github.dockerjava.api.exception.NotFoundException: Status 404: {"message":"Could not find the file /home/jenkins in container
1d303de7de9c6ea629291fc6ae01a7bfa4bb6d9a68de57bd23e763c784a80540"} at PluginClassLoader for docker-java-
api//com.github.dockerjava.core.DefaultInvocationBuilder.execute(DefaultInvocationBuilder.java:241) at PluginClassLoader for docker-java-
api//com.github.dockerjava.core.DefaultInvocationBuilder.put(DefaultInvocationBuilder.java:223) at PluginClassLoader for docker-java-
api//com.github.dockerjava.core.exec.CopyArchiveToContainerCmdExec.execute(CopyArchiveToContainerCmdExec.java:34) at PluginClassLoader for docker-java-
api//com.github.dockerjava.core.exec.CopyArchiveToContainerCmdExec.execute(CopyArchiveToContainerCmdExec.java:13) at PluginClassLoader for docker-java-
```

Php Pipeline

Here is an example how I would test and deploy an application with multiple containers (in this case php and mysql) In this way you need in the contraires what is needed for the application , Java for the Jenkins agent...

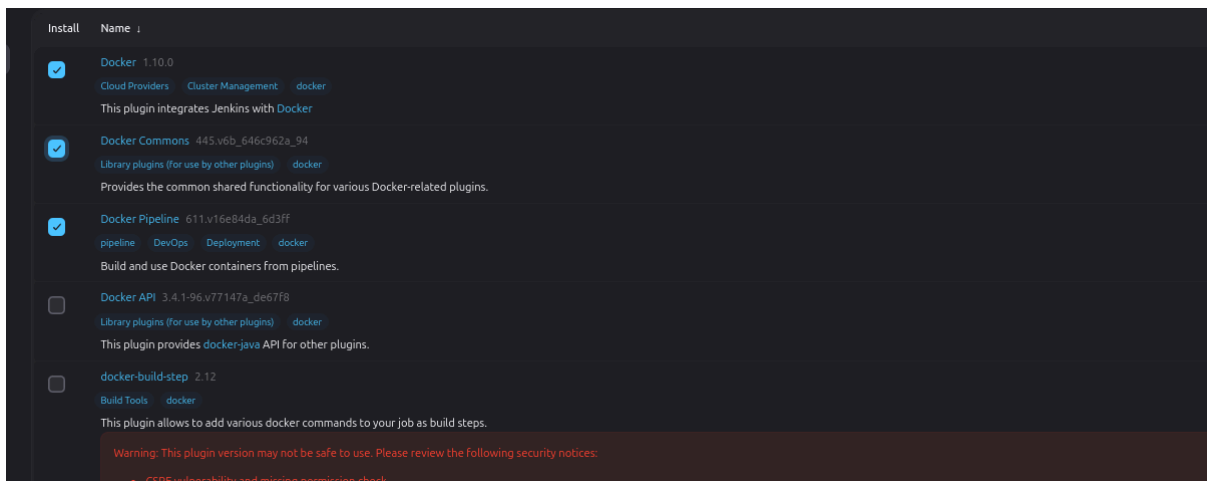
The best way for me to this actually as Free Style Jobs with pure shell scripts. See

Enable Docker

Verify that you have done in the host machine

```
sudo chmod 777 /var/run/docker.sock
```

Check that Docker plugins are installed.



Create php container

```
cd jenkinsPhp/
# delete jenkinsphp-php if it exists
docker image ls | grep 'jenkinsphp-php'
```

```
#jenkinsphp-php          latest          9d311f5dea81    2026
docker rmi IMAGE_ID_ABOVE
docker compose up -d
docker ps
#delete the obsolete container
docker stop jenkinsphp-php ;
docker rm jenkinsphp-php ;
```

Install git ftp

and configure the variables:

```
docker exec -it --user root jenkins_local apt-get install git-ftp
```

Define ftp credentials

This project deploys to my subdomain

<https://jenkins-php-pipeline.tantonius.com/> and the the git ftp must be configured for this.

This is not the most practical way, but a nice shell script. Actually you should make Jenkins credentials for this.

Open script `scripts/setEnvVariables.sh`

and configure the variables:

```
export
PHP_PIPELINE_FTP_HOST_URL=ftp.tantonius.com/domains/tantonius.com/public_html/jenkins-php-pipeline
export PHP_PIPELINE_FTP_USER=xxxx
export PHP_PIPELINE_FTP_PASSWORD=xxxxxxx
```

and run the script.

Create New pipeline

<http://localhost:8080/view/all/newJob>

New Item

Enter an item name

php-pipeline

Select an item type

- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project

php-pipeline

On page Config page <http://localhost:8080/job/php-pipeline/configure>

Select the following

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

1 Copu from https://github.com/cptuulia/jenkins/blob/main/jenkinsPhp/jenkinsfile try sample Pipeline...

☒ Use Groovy Sandbox ?

Pipeline Syntax

Copy jenkinsfile from

<https://github.com/cptuulia/jenkins/blob/main/jenkinsPhp/jenkinsfile>

And make sure the repo is public.

There is in the same folder a script jenkinsfile.sh which has the direct shell commands.

Use this before making a jenkins pipeline script...

Deploy

This project deploys to my subdomain
<https://jenkins-php-pipeline.tantonius.com/>

Make sure it exists.

PHP Free Style

This is the project I would use in the real world with my Jenkins skills.

We have three parts:

- 1) **Intialize GIT an subdomain <https://jenkins-php-pipeline.tantonius.com/>**
We pull the repo `https://github.com/cptuulia/jenkins` into a permanent repo,
which is also used by the job `01_job_php_shell_deploy`
This enables the deployment.
For this purpose we create a job: `job_php_initialize_git`
- 2) **Test**
Before each commit it must be tested. Later it can also inspected by PHPStan and
CodeSniffer etc.
For this purpose we create a job: `job_php_test`
- 3) **Deploy**
In this part we deploy the tested version to the live server.
Also I make a suggestion how I would migrate Laravel database changes.
For this purpose we create a job: `job_php_deploy`

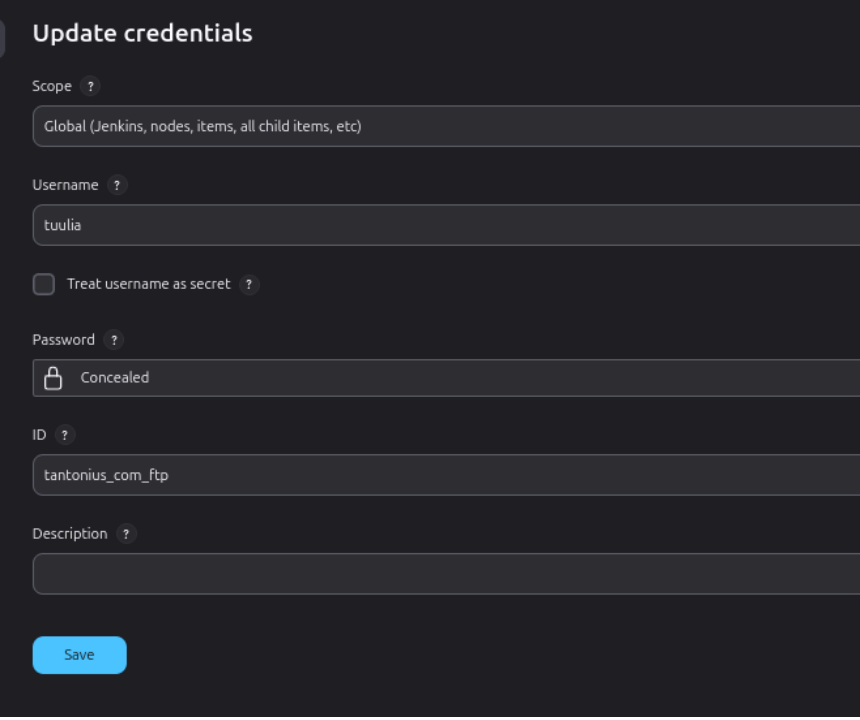
Subdomain jenkins-php-pipeline.tantonius.com

For this project I made a subdomain
<https://jenkins-php-pipeline.tantonius.com/>
Make sure it exists or make it again.

Define FTP Credentials

Define the FTP credentials to `fftp.tantonius.com`
as following:

Open <http://localhost:8080/manage/credentials/>
and add

A screenshot of the Jenkins 'Update credentials' form. The form has a dark theme. It includes fields for 'Scope' (set to 'Global (Jenkins, nodes, items, all child items, etc)'), 'Username' (set to 'tuulia'), a checkbox for 'Treat username as secret' (unchecked), 'Password' (set to 'Concealed'), 'ID' (set to 'tantonius_com_ftp'), and a 'Description' field. A blue 'Save' button is at the bottom.

id: tantonius_com_ftp

01 Initialize GIT for deployment

In this part we initialize the GIT repository on Jenkins server and on my subdomain
<https://jenkins-php-pipeline.tantonius.com/>

Git repo for deployment

On Jenkins server, make a repo folder as following:

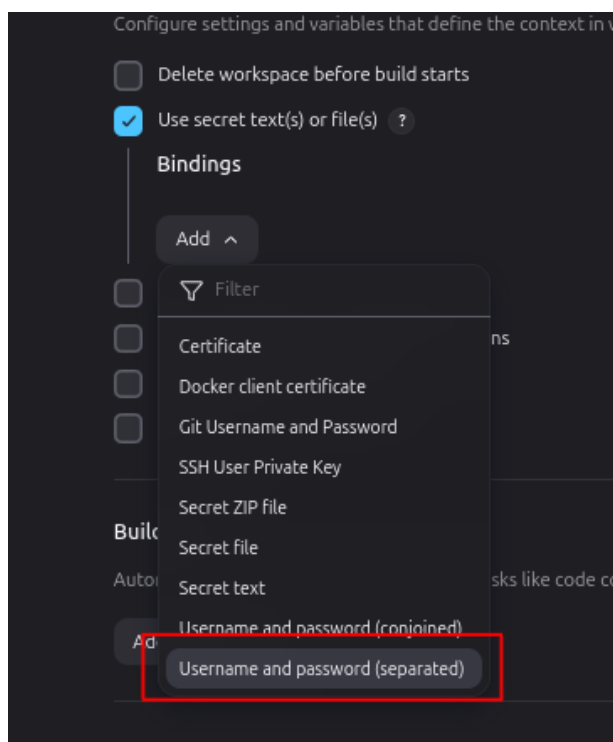
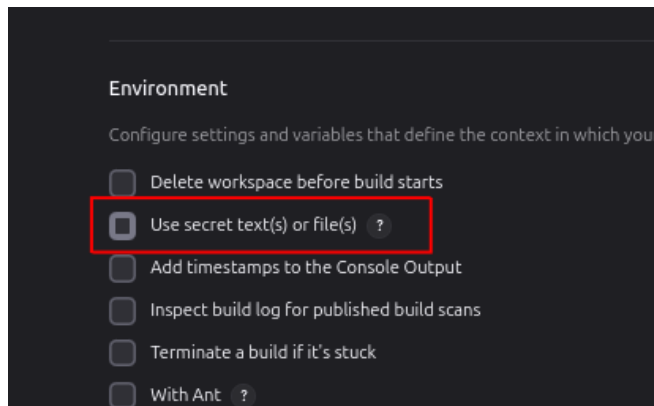
```
# to initialize run this in jenkins container, it is not recommended to run Free  
Style Jobs as root  
mkdir -p /var/jenkins_home/tuulia/php_pipeline/gitRepo  
chmod a+rw /var/jenkins_home/tuulia/php_pipeline/gitRepo
```

Job: job_php_initialize_git

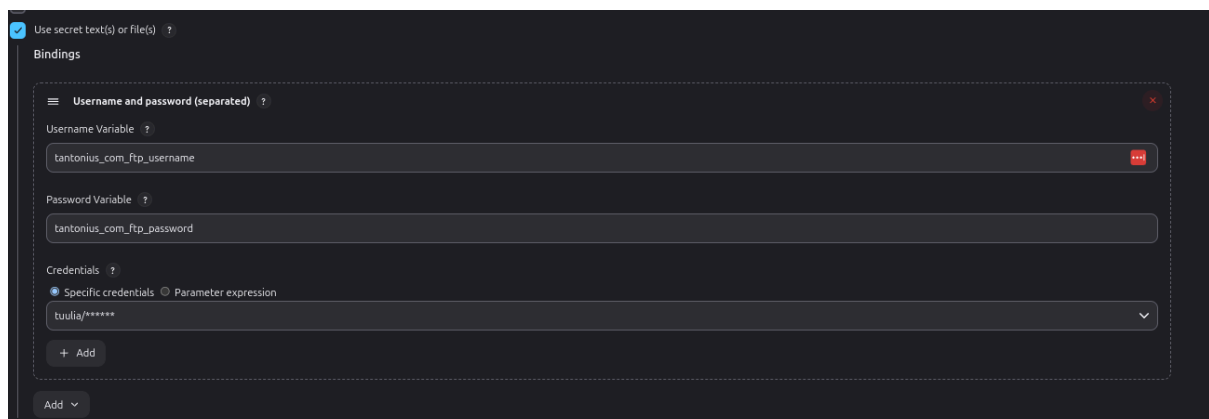
In this one time job we Initialize the git repo on Jenkins server and
<https://jenkins-php-pipeline.tantonius.com/>.

Make new freestyle job: [job_php_initialize_git](#)

Add a binding where you use the ftp credential we created

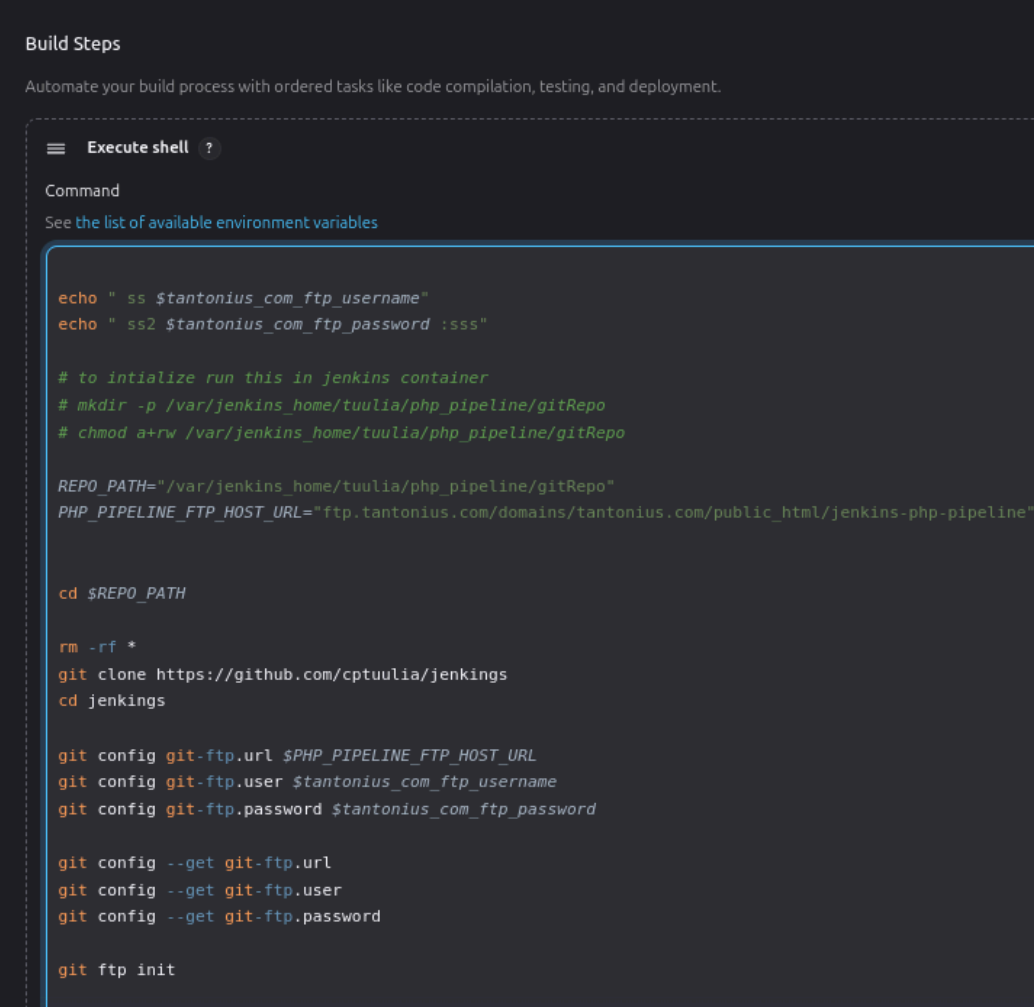


Just select:



tantonius_com_ftp_username
tantonius_com_ftp_password

Copy script `jenkinsPhp/01_job_php_shell_initialize_git.sh`



The screenshot shows the Jenkins 'Build Steps' configuration page. At the top, it says 'Build Steps' and 'Automate your build process with ordered tasks like code compilation, testing, and deployment.' Below this is a section for 'Execute shell' with a help icon. Under 'Command', there is a link 'See the list of available environment variables'. The main area contains a shell script for initializing a git repository and setting up ftp credentials.

```
echo " ss $stantonius_com_ftp_username"
echo " ss2 $stantonius_com_ftp_password :sss"

# to intialize run this in jenkins container
# mkdir -p /var/jenkins_home/tuulia/php_pipeline/gitRepo
# chmod a+rw /var/jenkins_home/tuulia/php_pipeline/gitRepo

REPO_PATH="/var/jenkins_home/tuulia/php_pipeline/gitRepo"
PHP_PIPELINE_FTP_HOST_URL="ftp.tantonius.com/domains/tantonius.com/public_html/jenkins-php-pipeline"

cd $REPO_PATH

rm -rf *
git clone https://github.com/cptuulia/jenkins
cd jenkins

git config git-ftp.url $PHP_PIPELINE_FTP_HOST_URL
git config git-ftp.user $stantonius_com_ftp_username
git config git-ftp.password $stantonius_com_ftp_password

git config --get git-ftp.url
git config --get git-ftp.user
git config --get git-ftp.password

git ftp init
```

02 Test

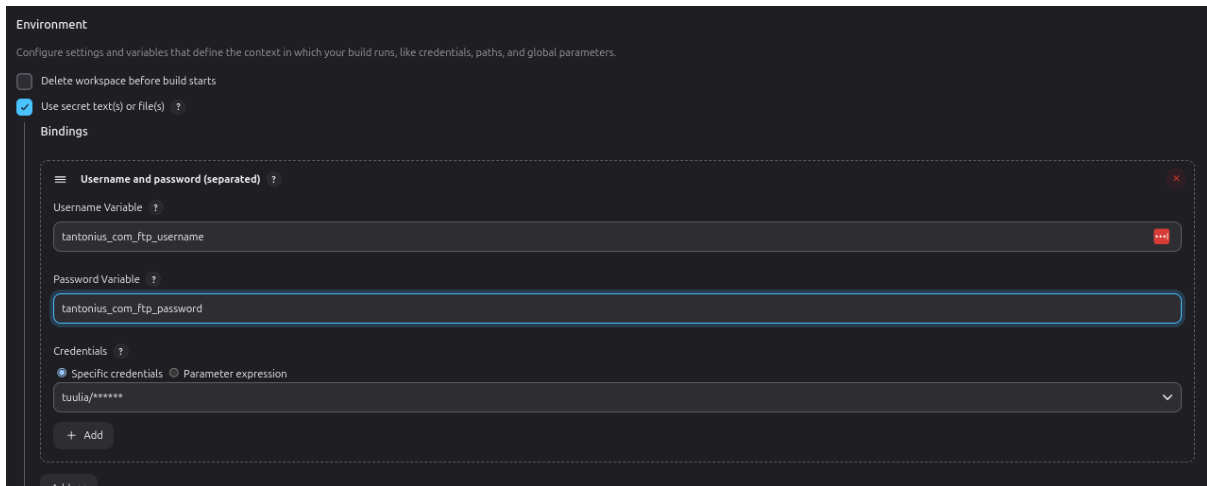
Make new freestyle job: [job_php_test](#)

And add the script from `jenkinsPhp/02_job_php_test.sh`

03 Deploy

Make new freestyle job: [job_php_deploy](#)

And add the ftp credential as you did with the job: [job_php_initialize_git](#)



tantonius_com_ftp_username
tantonius_com_ftp_password

Add the script `jenkinsPhp/03_job_php_deploy.sh`

Migrate

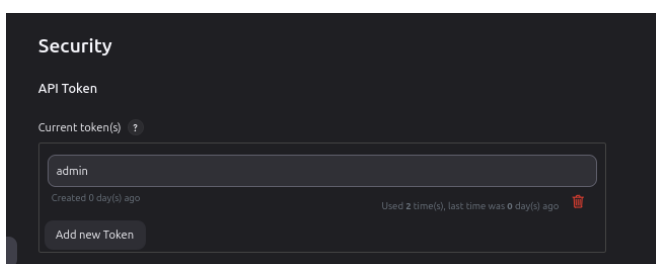
In case we want to migrate the database on the live server database with Jenkins by configuring it to remote connections to the live DB server.

Start build with CLI

1 Crate API token on page

<http://localhost:8080/user/admin/security/>

If this no more valid make new 113e32c863d0139cddd27e092f69ade149



Execute by

```
curl -u admin:<API_TOKEN> -FSubmit=Build 'http://localhost:8080/job/php-pipeline/build?delay=0sec'
```

I made a nice script for this. See

<https://github.com/cptuulia/jenkins/blob/main/scripts/buildPipeline.sh>

Push Docker Image

Here I push my image to my docker hub

Build images

run script and check that you have image `jenkingstestcontainer-php`

```
>./scripts/startTestContainer.sh
> docker image ls | grep 'jenkingstestcontainer-php'
jenkingstestcontainer-php          latest                24e23c369ad7        4 days ago          764MB
```

Test (nice to know)

<https://stackoverflow.com/questions/54182191/how-to-run-a-php-script-on-docker>

```
docker run -it --rm --name my-running-script -v "$PWD":/usr/src/myapp -w /usr/src/myapp
jenkingstestcontainer-php php jenkinsTestContainer/phpinfo.php
```

--rm will remove the container after execution
-v \$(pwd):/app/ will mount current directory
`jenkingstestcontainer-php` is the image

Push image

Here is how you push a docker image

- 1) Login to docker

```
docker login -u tuulia
```

- 2) Create a tag

```
docker tag jenkinsstestcontainer-php tuulia/jenkinsstestcontainer-php
```

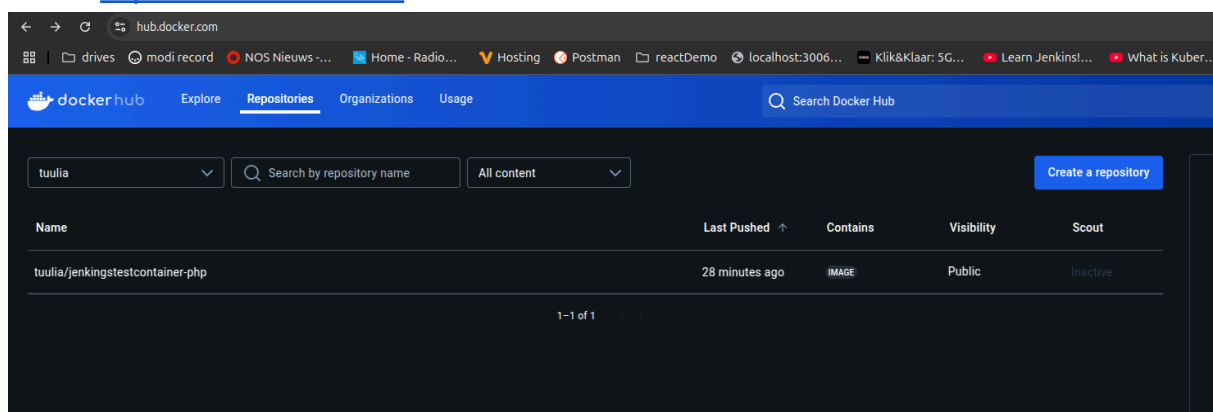
- 3) push

```
docker push tuulia/jenkinsstestcontainer-php
```

- 4) Check

Login as tuulia to

<https://hub.docker.com/>



<https://stackoverflow.com/questions/42898460/docker-push-data-container-onto-docker-hub>